Gimel Foundation

Gimel Foundation gGmbH i.G.
GiFo-Request for Comments: 0118
Obsoletes: -
Category: Standards Track
Version: 1.1

G. Wehberg, S. Raghuthaman and the Architecture Working Group of Gimel Foundation
10. April 2026

GAuth Management API & Administration Interface

Abstract

This specification defines the Management API and Administration Interface for the Gimel Authorization (GAuth) protocol with two layers:

- **Normative Core**: A Management API contract defining CRUD operations for mandate/PoA lifecycle management: creation with rule-based validation, querying and inspection, status transitions, suspension and resumption, revocation, budget management, TTL extension, delegation lifecycle, and governance profile assignment. All operations are deterministic and rule-based - no AI or probabilistic inference is required.
- **Non-Normative Guidance**: Administration Interface patterns for management UIs: reference screen specifications for mandate creation, inspection, delegation management, budget monitoring, and governance profile assignment.

Together, these complete the GAuth open core protocol triangle:

- RFC 0116: Credential Format | What does a PoA look like?
- RFC 0117: Enforcement | How do I check a PoA at runtime?
- RFC 0118: Management (this document) | How do I create, query, modify, and revoke a PoA?

This specification enables a purely rule-based, deterministic and AI-free registration path - alongside proprietary implementations may provide. A third-party implementing RFCs 0116, 0117, and 0118 can operate a complete GAuth-compatible authorization system without any proprietary dependencies. Any proprietary solutions (Exclusions) such as an AI-enabled governance, web3 integration as well as DNA based identities and PQC associated are not in scope of this specification. They do not fall under this open-source license but are subject to proprietary licensing by Gimel Foundation or Gimel Technologies, respectively.

Status of This Memo

This is a Gimel Foundation Standards Track document.

This document is a product of the Gimel Foundation (GiFo). It represents the current consensus of the Gimel Foundation community. It has performed review and has been approved for publication.

Information about the status of this document, any errata, and how to provide feedback on it may be obtained at https://gimelfoundation.com or https://github.com/Gimel-Foundation.

Legal notice

Copyright (c) 2026 Gimel Foundation and the persons identified as the document authors. All rights are reserved.

This document is subject to the Gimel Foundation's Legal Provisions Relating to GiFo Documents (see http://GimelFoundation.com or https://github.com/Gimel-Foundation) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document.

Code Components extracted from this document must include License text as described in Section 4. of the GiFo Legal Provisions Relating to Gimel Foundation Documents and are provided without warranty as described in the Provisions and its respective license conditions.

The distinguished GAuth standard as well as its management API and administrative interface are protected by copy right and patent law (patent pending). GAuth is an open-source standard, which can be based on OAuth, OpenID Connect and MCP. GAuth as well as its management API and administrative interface must not use Exclusions (see Scope under Section 2 of this Request for Comment), which are subject to separate license conditions and are also protected by copy right as well as patent law (patent pending).

Implementations of GAuth as well as this management API and administrative interface must reference to the Apache 2.0 license of OAuth and OpenID Connect as well as to the MIT license of MCP in line with its license conditions, as building blocks. Copyrights and licenses of these building blocks apply accordingly.

Implementations of GAuth as well as its management API and administrative interface must being licensed with Apache 2.0, granted by Gimel Foundation, and must not integrate Exclusions (as per Scope statement under Section 2 of this Request for Comment). Defined Exclusions of GAuth as well as of this management API and administrative interface must refer to separate license conditions.

Notational Conventions

The key words "Must", "Must Not", "Required", "Shall", "Shall Not", "Should", "Should Not", "Recommended", "May", and "Optional" in the following specification are to be interpreted as described in IETF`s RFC 2119.

* * *

Table of Content

1. Introduction

2. Legal Framework & Licensing

3. Architectural Overview

4. Management API — Mandate Lifecycle

5. Management API — Mandate Querying

6. Management API — Status Transitions

7. [Management API — Budget Operations](#7-management-api--budget-operations)

8. [Management API — Delegation Lifecycle](#8-management-api--delegation-lifecycle)

9. [Management API — Governance Profile Assignment](#9-management-api--governance-profile-assignment)

10. [Validation Semantics](#10-validation-semantics)

11. [HTTP Binding](#11-http-binding)

12. [Error Response Schema](#12-error-response-schema)

13. [Interface Versioning](#13-interface-versioning)

14. [Administration Interface Patterns (Non-Normative)](#14-administration-interface-patterns-non-normative)

15. [Conformance Requirements](#15-conformance-requirements)

16. [Security Considerations](#16-security-considerations)

Appendices:

- [A: Reference Implementation Pseudocode](#appendix-a-reference-implementation-pseudocode)

- [B: Management API Schema Registry](#appendix-b-management-api-schema-registry)

- [C: Administration Interface Wireframe Descriptions](#appendix-c-administration-interface-wireframe-descriptions)

- [D: Open Questions](#appendix-d-open-questions)

- [E: Reference Documents](#appendix-e-reference-documents)

- [F: Revision History](#appendix-f-revision-history)

---

1. INTRODUCTION

1.1 Purpose

GAuth defines a comprehensive authorization protocol for governing agent-based software systems. The protocol suite — RFCs 0110, 0111, 0115, 0116, and 0117 — specifies how PoA credentials are structured, issued, transported, validated, and enforced. However, it does not specify how those credentials are *managed* — i.e., how an administrator or automated system creates, inspects, modifies, and revokes mandates through a standardized interface.

Today, mandate lifecycle operations are handled by proprietary platform implementations. An AI-assisted system (such as G-Agent 1) may provide three registration paths with regulatory intelligence, risk scoring, and compliance reasoning. However, a third party building a GAuth-compatible system has no specification for the management operations — they can format a PoA (RFC 0116) and enforce it (RFC 0117), but cannot create or manage one through a published API contract.

This RFC provides that specification: a fourth registration path that is purely rule-based, deterministic, and AI-free. The caller provides all required mandate parameters; the Management API validates them against the PoA schema (RFC 0116) and governance

profile ceilings (RFC 0115); and the system either accepts the mandate or returns structured validation errors.

1.2 Design Principles

1. **Rule-based** — All validation is deterministic and schema-driven. The Management API does not require AI, probabilistic inference, or regulatory interpretation to function. Given the same input and the same governance profile ceilings, the API MUST produce the same acceptance/rejection result.

2. **Schema-validated** — Every mandate submitted through the Management API is validated against the canonical PoA JSON Schema defined in RFC 0116 §4.3. Schema validation is the first gate; ceiling validation is the second.

3. **Ceiling-enforced** — Mandate parameters are validated against the governance profile ceiling table (RFC 0115). No mandate may exceed the ceilings defined by its declared governance profile. Ceiling violations produce structured errors, not silent truncation.

4. **Fail-closed** — If the Management API cannot complete a validation (missing ceiling data, schema resolution failure, internal error), the mandate is rejected. There is no auto-approve mode.

5. **Auditable** — Every management operation (create, update, suspend, resume, revoke, delegate) produces an audit record. The audit trail is immutable and includes the identity of the caller, the operation performed, the mandate affected, and the validation result.

6. **Extendable** — Proprietary features may layer additional registration paths with AI-assisted validation, risk scoring, regulatory interpretation, or implied-authority mechanisms on top of this API. Such extensions operate outside the normative contract defined in this specification and are excluded from the Apache 2.0 license scope (see Section 2). This is consistent with the Extendable principle defined in RFC 0117 §1.2.

1.3 Terminology

Term	Definition
Management API	A standardized interface for creating, querying, modifying, and revoking PoA credentials (mandates)

| **Mandate** | A PoA credential as defined in RFC 0116: the complete authorization document specifying an agent's permissions, constraints, budget, and governance context |

| **Governance Profile** | One of five permission boundary definitions (minimal, standard, strict, enterprise, behörde) as defined in RFC 0115 |

| **Ceiling** | The maximum permitted value for a mandate attribute within a given governance profile |

| **Ceiling Violation** | A mandate attribute that exceeds the ceiling defined by the mandate's governance profile |

| **Registration Path** | A method by which a mandate is created and validated. This specification defines Path 4 (rule-based); Paths 1–3 are proprietary AI-assisted paths |

| **Status Transition** | A change in mandate lifecycle state (e.g., DRAFT → ACTIVE, ACTIVE → REVOKED) |

| **Suspension** | A temporary, reversible deactivation of an ACTIVE mandate. The mandate retains its identity, scope, budget, and audit trail. PEP enforcement rejects suspended mandates on CHK-02 (Temporal & Status). |

| **Administration Interface** | A visual user interface for managing mandates, delegations, budgets, and governance profiles |

1.4 Relationship to Prior RFCs

```
GiFo-RFC 0110 ──→ Protocol Engine (transport, routing, service mesh)
    │
GiFo-RFC 0111 ──→ Authorization Framework (flows, roles, trust model)
    │
GiFo-RFC 0115 ──→ Three-Layer Capability Model (Vollmacht conceptual schema)
    │           └─ Governance profile definitions and ceiling tables
    │
GiFo-RFC 0116 ──→ Interoperability Layer (token format, schemas, adapters)
    │           ├─ §4: PoA Credential Schema (validated by this document)
    │           ├─ §10: Token Lifecycle (operationalized by this document)
```

```
│           └─ §12: Error Response Schema (extended by this document)
│
GiFo-RFC 0117 ──→ Policy Enforcement Point (runtime enforcement)
│           ├─ §9: Evaluation Semantics (consumes mandates managed by this API)
│           └─ §10: Stateless/Stateful (mode impacts management operations)
│
GiFo-RFC 0118 ──→ Management API & Administration Interface (THIS DOCUMENT)
 (this doc)       ├─ §4-8: Management API Contract (CRUD, delegation, budget)
                  ├─ §9: Governance Profile Assignment
                  ├─ §10: Validation Semantics (rule-based, deterministic)
                  ├─ §11: HTTP Binding
                  └─ §14: Administration Interface Patterns (non-normative)
```

1.5 Scope

In Scope:

- Management API contract (mandate CRUD, delegation lifecycle, budget operations, TTL extension, suspension/resumption, governance profile assignment)
- Validation semantics (schema validation, ceiling enforcement, consistency checks)
- Budget consumption reporting interface (PEP → Management API)
- HTTP binding for Management API endpoints
- Error response schema for management operations
- Interface versioning
- Administration Interface patterns (non-normative guidance)

Out of Scope:

- AI-assisted registration paths (Paths 1–3 are proprietary; this specification defines Path 4 only)

- Risk scoring and composite risk assessment (proprietary value-add; see Extendable principle §1.2)

- Regulatory interpretation and compliance reasoning (proprietary; G-Agent 1 territory)

- Implied-authority mechanisms (proprietary; see RFC 0117 §1.2 Extendable principle)

- PEP enforcement semantics (defined in RFC 0117)

- Token format and transport (defined in RFC 0116)

- Billing and metering of management operations (internal platform concern)

- Governance profile ceiling definitions (defined in RFC 0115; this specification validates against them)

1.6 The Four Registration Paths

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    MANDATE REGISTRATION PATHS                            │
│                                                                          │
│  Path 1 – AI-Assisted (Behörde/Public Sector)   ┐                        │
│  Path 2 – AI-Assisted (Enterprise)              ├– Proprietary           │
│  Path 3 – AI-Assisted (Mittelstand/Startup/Proto) ┘ (G-Agent 1)          │
│                                                                          │
│  Path 4 – Rule-Based (Deterministic, No AI)     — Open (RFC 0118)        │
│      │                                                                   │
│      ├– Schema validation (RFC 0116 §4.3)                                │
│      ├– Ceiling enforcement (RFC 0115)                                   │
│      ├– Consistency checks (6 rules)                                     │
│      └– Structured error responses                                       │
│                                                                          │
│  Path 4 is the foundation — Paths 1–3 layer AI enhancement on top.       │
│  Path 4 validation always runs; Paths 1–3 add risk scoring,              │
```

```
│ regulatory analysis, and implied-authority reasoning.              │
│                                                                     │
│ All four paths produce RFC 0116-conformant PoA credentials that are │
│ enforced identically by RFC 0117-conformant PEPs.                   │
└─────────────────────────────────────────────────────────────────────┘
```

---

2. LEGAL FRAMEWORK & LICENSING

2.1 Apache License 2.0 Scope

GiFo-RFC 0118 is published under the Apache License, Version 2.0. The Management API contract, validation semantics, and reference implementation pseudocode are fully open.

In Scope (Open Source — Apache 2.0):

Component	Description
Management API Contract (Sections 4-9)	CRUD operations, delegation lifecycle, budget operations, TTL extension, suspension/resumption, governance profile assignment
Validation Semantics (Section 10)	Schema validation, ceiling enforcement, consistency checks
HTTP Binding (Section 11)	RESTful endpoint definitions and request/response formats
Administration Interface Patterns (Section 14)	Non-normative UI guidance
Reference Implementation (Appendix A)	Language-agnostic pseudocode

Out of Scope (Not part of this specification):

| Component | Description |

|----------|----------|

| AI-assisted registration paths | Risk scoring, regulatory interpretation, compliance reasoning — these are proprietary and not part of the open Management API contract |

| Proprietary management extensions | Implied-authority resolution, organizational policy engines, custom approval workflows — platform-specific |

2.2 Contributor License Agreement

Contributors to GiFo-RFC 0118 grant the Gimel Foundation a perpetual, worldwide, non-exclusive, royalty-free license to use, reproduce, modify, and distribute their contributions under the Apache License 2.0.

2.3 Patent Grant

Subject to the terms and conditions of the Apache License 2.0, each contributor grants to the Gimel Foundation and to recipients of the specification a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable patent license to make, use, sell, and otherwise transfer implementations that conform to this specification.

---

3. ARCHITECTURAL OVERVIEW

3.1 Management API in the GAuth Architecture

The Management API sits between the administrator (or automated provisioning system) and the mandate store. It governs how mandates enter and exit the system, while the PEP (RFC 0117) governs how they are enforced at runtime.

```
┌──────────────────────────────────────────────────────────────────────────┐
│ Administrator / Provisioning System                                      │
│                                                                          │
│ 1. Submit mandate creation request                                       │
│ 2. Provide all required parameters (scope, budget, governance profile) │
└──────────────────────┬──────────────────────────────────────────────────┘
                       │ Management Request
                       ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ MANAGEMENT API — RFC 0118                                                │
│                                                                          │
│ ┌──────────────────────────────────────────────────────────────────────┐ │
│ │ 1. Schema Validation (RFC 0116 §4.3)                                 │ │
│ │ 2. Ceiling Enforcement (RFC 0115 governance profile ceilings)        │ │
│ │ 3. Consistency Checks (6 deterministic rules)                        │ │
│ │ 4. Scope Checksum Computation                                        │ │
│ │ 5. Audit Record Generation                                           │ │
│ └──────────────────────────────────────────────────────────────────────┘ │
│                                                                          │
│ Result: ACCEPTED (mandate stored) or REJECTED (structured errors)        │
└──────────────────────┬──────────────────────────────────────────────────┘
                       │
                       ▼
```

```
┌──────────────────────────────────────────────────────────────────────┐
│ MANDATE STORE                                                        │
│                                                                      │
│ Persisted mandate available for:                                     │
│ • PEP enforcement (RFC 0117) — stateful mode reads live mandate state │
│ • PEP budget reporting — consumption deductions written back to store │
│ • Token issuance (RFC 0116) — OAuth engine embeds PoA in JWT/VC      │
│ • Administration Interface — query, inspect, modify                  │
└──────────────────────────────────────────────────────────────────────┘
```

3.2 Mandate Lifecycle States

Mandates follow a defined lifecycle with deterministic state transitions:

```
              ┌──────────┐
              │ DRAFT    │
              └────┬─────┘
                   │ activate()
                   ▼
              ┌──────────┐
         ┌────│ ACTIVE   │────┬──────────┐
         │    └────┬─────┘    │          │
         │         │          │ suspend()
   revoke() │ expire() │ exhaust() │          │
         │         │ budget   │          ▼
         │         │          │  ┌──────────┐
```

```
        │      │      │  │ SUSPENDED │
        │      │      │  └─────┬─────┘
        │      │      │        │ resume()
        │      │      │        │ (→ ACTIVE)
        ▼      ▼      ▼        │
    ┌─────────┐ ┌─────────┐ ┌──────────────┐ ┌────────────┐
    │ REVOKED │ │ EXPIRED │ │BUDGET_EXCEEDED│ │ SUPERSEDED │
    └─────────┘ └─────────┘ └──────────────┘ └────────────┘
```

State	Description	Terminal
`DRAFT`	Mandate created but not yet active. May be modified. Not enforceable by PEP.	No
`ACTIVE`	Mandate is live and enforceable. Budget is being consumed. Scope is immutable.	No
`SUSPENDED`	Mandate temporarily deactivated. PEP rejects enforcement requests (CHK-02). Scope, budget, and audit trail are preserved. May be resumed to ACTIVE or revoked to terminal. TTL clock continues during suspension.	No
`EXPIRED`	TTL has elapsed. Automatically transitioned by the system.	Yes
`REVOKED`	Explicitly revoked by an authorized party.	Yes
`BUDGET_EXCEEDED`	Budget fully consumed. Automatically transitioned by the system.	Yes
`SUPERSEDED`	Replaced by a newer mandate for the same agent + project.	Yes

Transition rules:

- `DRAFT → ACTIVE`: Requires successful validation (schema + ceiling + consistency). Irreversible once scope is committed.
- `ACTIVE → SUSPENDED`: Requires caller identity with suspension authority (issuer or approver). Immediate effect — PEP enforcement rejects on next check.

- `SUSPENDED → ACTIVE`: Requires caller identity with resumption authority (issuer or approver). Mandate resumes enforcement. TTL is not extended by suspension duration.

- `SUSPENDED → REVOKED`: A suspended mandate may be revoked directly without resuming first.

- `ACTIVE → REVOKED`: Requires caller identity with revocation authority. Immediate effect — PEP enforcement rejects on next check.

- `ACTIVE → EXPIRED`: System-initiated when `ttl_seconds` elapses from activation time. Also applies to SUSPENDED mandates — a suspended mandate whose TTL elapses transitions directly to EXPIRED.

- `ACTIVE → BUDGET_EXCEEDED`: System-initiated when `remaining_cents` reaches zero.

- `ACTIVE → SUPERSEDED`: System-initiated when a new mandate is activated for the same `(agent_id, project_id)` pair. This transition MUST be atomic — see §6.4.

- Terminal states are irreversible. No transition FROM a terminal state is permitted.

Suspension rationale: Suspension enables temporary deactivation during security investigations, compliance reviews, or administrative holds without destroying the mandate's identity, scope, budget state, or audit trail. The alternative — revoke and re-create — loses continuity and requires re-validation.

3.3 Mutability Rules

Mandate State	Scope	Budget Ceiling	TTL	Status	Metadata
`DRAFT`	Mutable	Mutable	Mutable	Mutable (→ ACTIVE only)	Mutable
`ACTIVE`	**Immutable**	Additive-only (ceiling may increase, never decrease)	Additive-only (TTL may be extended, never shortened)	Mutable (→ SUSPENDED, terminal states)	Read-only
`SUSPENDED`	**Immutable**	Additive-only	Additive-only	Mutable (→ ACTIVE, REVOKED, EXPIRED)	Read-only
Terminal	Immutable	Frozen	Frozen	Frozen	Read-only

Scope immutability rationale: Once a mandate is active, its scope (governance_profile, core_verbs, platform_permissions, allowed_paths, denied_paths, phase, active_modules) is frozen. This ensures that the PEP's enforcement decisions are stable — a mandate cannot be weakened while in use. Budget ceiling and TTL are the exceptions: they may be increased (additive-only) to extend a mandate's operational life without changing its permissions.

---

4. MANAGEMENT API — MANDATE LIFECYCLE

4.1 Create Mandate

Creates a new mandate in `DRAFT` state.

Method signature (language-agnostic):

```
createMandate(request: MandateCreationRequest) → MandateCreationResponse
```

MandateCreationRequest schema:

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://gimelfoundation.com/schemas/mgmt/v1.1/mandate-creation-request.json",
  "title": "GAuth Management API — Mandate Creation Request",
  "type": "object",
```

```
 "required": ["parties", "scope", "requirements"],
 "properties": {
  "parties": {
   "type": "object",
   "required": ["subject", "customer_id", "project_id", "issued_by"],
   "properties": {
    "subject": {
     "type": "string",
     "description": "Agent ID or DID of the agent receiving the mandate"
    },
    "customer_id": {
     "type": "string",
     "description": "Customer identifier"
    },
    "project_id": {
     "type": "string",
     "description": "Project identifier within the customer context"
    },
    "issued_by": {
     "type": "string",
     "description": "Identity of the human or system creating the mandate"
    },
    "approval_chain": {
     "type": "array",
     "items": { "type": "string" },
     "description": "For four-eyes approval: ordered list of approver identities"
    }
   }
  },
```

```
  "scope": {
    "type": "object",
    "required": ["governance_profile", "phase", "core_verbs"],
    "properties": {
      "governance_profile": {
        "type": "string",
        "enum": ["minimal", "standard", "strict", "enterprise", "behoerde"]
      },
      "active_modules": {
        "type": "array",
        "items": { "type": "string" }
      },
      "phase": {
        "type": "string",
        "enum": ["plan", "build", "run"]
      },
      "allowed_paths": {
        "type": "array",
        "items": { "type": "string" }
      },
      "denied_paths": {
        "type": "array",
        "items": { "type": "string" }
      },
      "allowed_sectors": {
        "type": "array",
        "items": { "type": "string" },
        "description": "NAICS-based sector codes"
      },
```

```
      "allowed_regions": {
        "type": "array",
        "items": { "type": "string" },
        "description": "ISO 3166-1 + supranational identifiers"
      },
      "core_verbs": {
        "type": "object",
        "description": "Layer 1: Core action verbs with Layer 2 permission policies. Conforms to RFC 0116 §4.3 ToolPolicy schema.",
        "additionalProperties": {
          "$ref": "https://gimelfoundation.com/schemas/poa/v2.2/poa-credential.json#/$defs/ToolPolicy"
        }
      },
      "platform_permissions": {
        "$ref": "https://gimelfoundation.com/schemas/poa/v2.2/poa-credential.json#/$defs/PlatformPermissions",
        "description": "Layer 3: Agent-based platform capabilities"
      }
    }
  },
  "requirements": {
    "type": "object",
    "required": ["approval_mode", "budget", "ttl_seconds"],
    "properties": {
      "approval_mode": {
        "type": "string",
        "enum": ["autonomous", "supervised", "four-eyes"]
      },
```

```
      "budget": {
        "type": "object",
        "required": ["total_cents"],
        "properties": {
          "total_cents": {
            "type": "integer",
            "minimum": 0,
            "description": "Budget ceiling in cents"
          }
        }
      },
      "ttl_seconds": {
        "type": "integer",
        "minimum": 60,
        "description": "Time-to-live in seconds from activation"
      },
      "session_limits": {
        "type": "object",
        "properties": {
          "max_tool_calls": { "type": "integer", "minimum": 1 },
          "max_session_duration_minutes": { "type": "integer", "minimum": 1 },
          "max_lines_per_commit": { "type": "integer", "minimum": 1 }
        }
      }
    }
  }
 }
}
```

Behavior:

1. Validate request through the three-stage pipeline (Section 10)

2. Compute `scope_checksum` (SHA-256 of canonical JSON serialization of the `scope` object)

3. Compute `tool_permissions_hash` (SHA-256 of canonical JSON serialization of `core_verbs`)

4. Compute `platform_permissions_hash` (SHA-256 of canonical JSON serialization of `platform_permissions`)

5. Generate mandate ID (UUID v4 or equivalent — MUST be unpredictable, no sequential IDs)

6. Store mandate in `DRAFT` state

7. Generate audit record

MandateCreationResponse:

```json
{
  "mandate_id": "mdt_abc123",
  "status": "DRAFT",
  "scope_checksum": "sha256:a1b2c3d4...",
  "tool_permissions_hash": "sha256:4a5b6c7d...",
  "platform_permissions_hash": "sha256:1a2b3c4d...",
  "created_at": "2026-04-08T10:00:00Z",
  "validation": {
    "accepted": true,
    "schema_errors": [],
    "ceiling_violations": [],
    "consistency_errors": []
  },
```

```
  "audit": {
    "operation": "CREATE",
    "performed_by": "user_789",
    "timestamp": "2026-04-08T10:00:00Z",
    "mandate_id": "mdt_abc123"
  }
}
```

4.2 Activate Mandate

Transitions a `DRAFT` mandate to `ACTIVE`. Re-runs the full validation pipeline (Section 10) to ensure the mandate is still valid at activation time.

Method signature:

```
activateMandate(request: MandateActivationRequest) → MandateActivationResponse
```

MandateActivationRequest:

```json
{
  "mandate_id": "mdt_abc123",
  "activated_by": "user_789"
}
```

Behavior:

1. Verify mandate is in `DRAFT` state (reject if not)

2. Re-run three-stage validation pipeline (Section 10) — mandate parameters may have become invalid since creation (e.g., governance profile ceilings updated)

3. Check for existing `ACTIVE` mandate for the same `(agent_id, project_id)` pair

4. If existing active mandate found: atomically supersede it (§6.4) before activating this one

5. Set `activated_at` timestamp and compute `expires_at` = `activated_at` + `ttl_seconds`

6. Transition status to `ACTIVE`

7. Generate audit record

MandateActivationResponse:

```json
{
  "mandate_id": "mdt_abc123",
  "status": "ACTIVE",
  "activated_at": "2026-04-08T10:05:00Z",
  "expires_at": "2026-04-08T22:05:00Z",
  "superseded_mandate_id": null,
  "audit": {
    "operation": "ACTIVATE",
    "performed_by": "user_789",
    "timestamp": "2026-04-08T10:05:00Z",
    "mandate_id": "mdt_abc123"
  }
}
```

4.3 Revoke Mandate

Transitions an `ACTIVE` or `SUSPENDED` mandate to `REVOKED`. Immediate effect — the PEP will reject enforcement requests on the next check (CHK-02 temporal & status, per RFC 0117 §9.2).

Method signature:

```
revokeMandate(request: MandateRevocationRequest) → MandateRevocationResponse
```

MandateRevocationRequest:

```json
{
  "mandate_id": "mdt_abc123",
  "revoked_by": "user_789",
  "reason": "Security incident — agent compromised"
}
```

Behavior:

1. Verify mandate is in `ACTIVE` or `SUSPENDED` state (reject if terminal)
2. Verify caller has revocation authority (must be `issued_by` or a member of `approval_chain`)
3. Transition status to `REVOKED`
4. Cascade revocation to all child mandates in the delegation chain (§8.2)

5. Generate audit record with reason

MandateRevocationResponse:

```json
{
  "mandate_id": "mdt_abc123",
  "status": "REVOKED",
  "revoked_at": "2026-04-08T11:00:00Z",
  "revoked_by": "user_789",
  "reason": "Security incident — agent compromised",
  "cascaded_revocations": [],
  "audit": {
    "operation": "REVOKE",
    "performed_by": "user_789",
    "timestamp": "2026-04-08T11:00:00Z",
    "mandate_id": "mdt_abc123"
  }
}
```

4.4 Suspend Mandate

Transitions an `ACTIVE` mandate to `SUSPENDED`. The mandate retains its identity, scope, budget, and audit trail. PEP enforcement rejects suspended mandates on CHK-02 (Temporal & Status).

Method signature:

```
suspendMandate(request: MandateSuspensionRequest) → MandateSuspensionResponse
```

MandateSuspensionRequest:

```json
{
  "mandate_id": "mdt_abc123",
  "suspended_by": "user_789",
  "reason": "Security investigation — under review"
}
```

Behavior:

1. Verify mandate is in `ACTIVE` state (reject if not)
2. Verify caller has suspension authority (must be `issued_by` or a member of `approval_chain`)
3. Transition status to `SUSPENDED`
4. Cascade suspension to all child mandates in the delegation chain
5. Generate audit record with reason

MandateSuspensionResponse:

```json
{
  "mandate_id": "mdt_abc123",
  "status": "SUSPENDED",
```

```
  "suspended_at": "2026-04-08T13:00:00Z",
  "suspended_by": "user_789",
  "reason": "Security investigation — under review",
  "cascaded_suspensions": [],
  "audit": {
    "operation": "SUSPEND",
    "performed_by": "user_789",
    "timestamp": "2026-04-08T13:00:00Z",
    "mandate_id": "mdt_abc123"
  }
}
```

TTL behavior during suspension: The TTL clock continues during suspension. A mandate that expires while suspended transitions directly to `EXPIRED`. Suspension does not extend the mandate's lifetime — use `extendTTL()` (§7.4) if additional time is needed after resumption.

4.5 Resume Mandate

Transitions a `SUSPENDED` mandate back to `ACTIVE`. The mandate resumes enforcement with its original scope, remaining budget, and remaining TTL.

Method signature:

```
resumeMandate(request: MandateResumptionRequest) → MandateResumptionResponse
```

MandateResumptionRequest:

```json
{
  "mandate_id": "mdt_abc123",
  "resumed_by": "user_789",
  "reason": "Investigation complete — no issues found"
}
```

Behavior:

1. Verify mandate is in `SUSPENDED` state (reject if not)
2. Verify mandate has not expired during suspension (reject with `MANDATE_EXPIRED` if TTL elapsed)
3. Verify caller has resumption authority (must be `issued_by` or a member of `approval_chain`)
4. Transition status to `ACTIVE`
5. Cascade resumption to all child mandates that were suspended via cascade
6. Generate audit record with reason

MandateResumptionResponse:

```json
{
  "mandate_id": "mdt_abc123",
  "status": "ACTIVE",
  "resumed_at": "2026-04-08T15:00:00Z",
  "resumed_by": "user_789",
  "remaining_ttl_seconds": 25200,
```

```
  "audit": {
    "operation": "RESUME",
    "performed_by": "user_789",
    "timestamp": "2026-04-08T15:00:00Z",
    "mandate_id": "mdt_abc123"
  }
}
```

---

5. MANAGEMENT API — MANDATE QUERYING

5.1 Get Mandate

Retrieves a single mandate by ID.

Method signature:

```
getMandate(mandate_id: string) → MandateDetail
```

MandateDetail:

```json
{
  "mandate_id": "mdt_abc123",
  "status": "ACTIVE",
```

```
 "parties": {
  "issuer": "https://gimelfoundation.com/gauth",
  "subject": "agent_456",
  "customer_id": "cust_123",
  "project_id": "proj_789",
  "issued_by": "user_789"
 },
 "scope": {
  "governance_profile": "standard",
  "active_modules": ["security-basics", "data-model-first"],
  "phase": "build",
  "allowed_paths": ["src/", "tests/"],
  "denied_paths": [".env", "secrets/"],
  "core_verbs": { },
  "platform_permissions": { }
 },
 "requirements": {
  "approval_mode": "supervised",
  "budget": {
   "total_cents": 10000,
   "remaining_cents": 4200,
   "consumed_cents": 5800,
   "utilization_percent": 58.0
  },
  "session_limits": {
   "max_tool_calls": 500,
   "max_session_duration_minutes": 240,
   "max_lines_per_commit": 500
  },
```

```
  "ttl_seconds": 43200,
  "remaining_ttl_seconds": 25200
 },
 "scope_checksum": "sha256:a1b2c3d4...",
 "tool_permissions_hash": "sha256:4a5b6c7d...",
 "platform_permissions_hash": "sha256:1a2b3c4d...",
 "created_at": "2026-04-08T10:00:00Z",
 "activated_at": "2026-04-08T10:05:00Z",
 "expires_at": "2026-04-08T22:05:00Z",
 "delegation_chain": []
}
```

5.2 List Mandates

Queries mandates with filtering and pagination.

Method signature:

```
listMandates(query: MandateQuery) → MandateListResponse
```

MandateQuery:

```json
{
 "filter": {
  "customer_id": "cust_123",
```

```
    "project_id": "proj_789",
    "agent_id": "agent_456",
    "status": ["ACTIVE", "DRAFT", "SUSPENDED"],
    "governance_profile": "standard",
    "created_after": "2026-04-01T00:00:00Z",
    "created_before": "2026-04-08T23:59:59Z"
  },
  "pagination": {
    "cursor": null,
    "limit": 50
  },
  "sort": {
    "field": "created_at",
    "order": "desc"
  }
}
```

MandateListResponse:

```json
{
  "mandates": [ ],
  "pagination": {
    "next_cursor": "eyJpZCI6Im1kdF94eXoifQ==",
    "has_more": true,
    "total_count": 142
  }
}
```

```

### 5.3 Get Mandate History

Retrieves the full audit trail for a mandate.

**Method signature:**

```
getMandateHistory(mandate_id: string) → MandateHistoryResponse
```

**MandateHistoryResponse:**

```json
{
  "mandate_id": "mdt_abc123",
  "events": [
    {
      "event_type": "CREATE",
      "timestamp": "2026-04-08T10:00:00Z",
      "performed_by": "user_789",
      "details": { "initial_status": "DRAFT" }
    },
    {
      "event_type": "ACTIVATE",
      "timestamp": "2026-04-08T10:05:00Z",
      "performed_by": "user_789",
      "details": { "superseded_mandate_id": null }
```
```

```
  },
  {
    "event_type": "BUDGET_CONSUMPTION",
    "timestamp": "2026-04-08T12:30:00Z",
    "performed_by": "pep:enforcement",
    "details": { "deducted_cents": 42, "remaining_cents": 9958,
"enforcement_request_id": "enf_xyz" }
  },
  {
    "event_type": "BUDGET_INCREASE",
    "timestamp": "2026-04-08T14:30:00Z",
    "performed_by": "user_789",
    "details": { "previous_ceiling": 10000, "new_ceiling": 15000 }
  },
  {
    "event_type": "SUSPEND",
    "timestamp": "2026-04-08T16:00:00Z",
    "performed_by": "user_789",
    "details": { "reason": "Security investigation" }
  },
  {
    "event_type": "RESUME",
    "timestamp": "2026-04-08T18:00:00Z",
    "performed_by": "user_789",
    "details": { "reason": "Investigation complete" }
  }
 ]
}
```

---

6. MANAGEMENT API — STATUS TRANSITIONS

6.1 Transition Rules

Status transitions are governed by the following state machine. The Management API MUST enforce these rules; any invalid transition MUST return an error.

From State	To State	Trigger	Authority Required
`DRAFT`	`ACTIVE`	`activateMandate()`	Mandate creator or approver
`DRAFT`	(deleted)	`deleteMandate()`	Mandate creator
`ACTIVE`	`SUSPENDED`	`suspendMandate()`	Issuer, approver, or system
`ACTIVE`	`REVOKED`	`revokeMandate()`	Issuer, approver, or delegated revocation authority
`ACTIVE`	`EXPIRED`	System clock (TTL elapsed)	Automatic
`ACTIVE`	`BUDGET_EXCEEDED`	System (remaining_cents = 0)	Automatic
`ACTIVE`	`SUPERSEDED`	New mandate activated for same (agent, project)	Automatic
`SUSPENDED`	`ACTIVE`	`resumeMandate()`	Issuer or approver
`SUSPENDED`	`REVOKED`	`revokeMandate()`	Issuer, approver, or delegated revocation authority
`SUSPENDED`	`EXPIRED`	System clock (TTL elapsed during suspension)	Automatic

6.2 Draft Deletion

DRAFT mandates that have not been activated may be deleted. This is a hard delete — the mandate is removed from the store. An audit record is retained.

Method signature:

```
deleteMandate(mandate_id: string, deleted_by: string) → DeletionConfirmation
```

Constraints:

- Only `DRAFT` mandates may be deleted
- Only the mandate creator (`issued_by`) may delete
- Audit record is retained even after deletion

6.3 Automatic Transitions

The Management API implementation MUST handle automatic transitions:

1. **Expiration:** A background process or lazy check MUST transition `ACTIVE` and `SUSPENDED` mandates to `EXPIRED` when `activated_at + ttl_seconds < current_time`. PEP implementations performing stateful enforcement (RFC 0117 §10.3) MUST also check temporal validity and treat expired mandates as `EXPIRED` even if the store has not yet transitioned the status.

2. **Budget exhaustion:** When a budget deduction reduces `remaining_cents` to zero, the mandate MUST transition to `BUDGET_EXCEEDED`. PEP implementations checking budget (CHK-13) MUST also enforce this boundary.

3. **Supersession:** When `activateMandate()` detects an existing `ACTIVE` mandate for the same `(agent_id, project_id)` pair, the prior mandate MUST transition to `SUPERSEDED` before the new mandate is activated. See §6.4 for atomicity requirements.

6.4 Supersession Atomicity

The supersession transition (ACTIVE → SUPERSEDED + new DRAFT → ACTIVE) MUST be atomic. Implementations MUST ensure that at no point are two mandates simultaneously in `ACTIVE` state for the same `(agent_id, project_id)` pair.

Implementation requirements:

- The supersession and activation MUST be performed within a single transaction or under an exclusive lock on the `(agent_id, project_id)` pair

- If the transaction fails, neither the supersession nor the activation occurs (rollback)

- Concurrent `activateMandate()` calls for the same `(agent_id, project_id)` pair MUST be serialized — the second caller receives `DUPLICATE_MANDATE` if both target the same pair

Rationale: Without atomicity, a race condition between two concurrent activations could leave both mandates active, or supersede the wrong mandate. The PEP (RFC 0117 CHK-02) expects exactly zero or one active mandate per `(agent_id, project_id)` pair in stateful mode.

---

7. MANAGEMENT API — BUDGET OPERATIONS

7.1 Budget Model

Each mandate has a budget defined in cents:

Field	Type	Mutability	Description

| `total_cents` | integer | Additive-only (ACTIVE/SUSPENDED); mutable (DRAFT) | Budget ceiling — may only increase once active |

| `remaining_cents` | integer | Decremented by PEP enforcement and budget consumption reports | Current balance |

| `consumed_cents` | integer (computed) | Read-only | `total_cents - remaining_cents` |

7.2 Increase Budget Ceiling

Increases the budget ceiling of an `ACTIVE` or `SUSPENDED` mandate. The budget ceiling may only increase, never decrease (additive-only rule).

Method signature:

```
increaseBudget(request: BudgetIncreaseRequest) → BudgetIncreaseResponse
```

BudgetIncreaseRequest:

```json
{
  "mandate_id": "mdt_abc123",
  "additional_cents": 5000,
  "increased_by": "user_789",
  "reason": "Extended project scope"
}
```

Behavior:

1. Verify mandate is `ACTIVE` or `SUSPENDED`

2. Verify `additional_cents` > 0

3. Add `additional_cents` to `total_cents` and `remaining_cents` atomically

4. Generate audit record

5. Return updated budget state

BudgetIncreaseResponse:

```json
{
  "mandate_id": "mdt_abc123",
  "budget": {
    "total_cents": 15000,
    "remaining_cents": 9200,
    "consumed_cents": 5800
  },
  "audit": {
    "operation": "BUDGET_INCREASE",
    "performed_by": "user_789",
    "timestamp": "2026-04-08T14:30:00Z",
    "mandate_id": "mdt_abc123",
    "details": { "previous_ceiling": 10000, "new_ceiling": 15000 }
  }
}
```

7.3 Report Budget Consumption

PEP implementations and enforcement systems report budget consumption back to the mandate store through the Management API. This is the normative interface for keeping budget state synchronized between the enforcement layer and the management layer.

Method signature:

```
reportConsumption(request: ConsumptionReport) → ConsumptionAcknowledgement
```

ConsumptionReport:

```json
{
  "mandate_id": "mdt_abc123",
  "deducted_cents": 42,
  "enforcement_request_id": "enf_xyz789",
  "operation_type": "agent_command_authorization",
  "timestamp": "2026-04-08T12:30:00Z",
  "reported_by": "pep:enforcement"
}
```

Behavior:

1. Verify mandate is `ACTIVE` (reject consumption reports for non-active mandates)
2. Verify `deducted_cents` > 0
3. Atomically decrement `remaining_cents` by `deducted_cents`
4. If `remaining_cents` reaches zero: transition mandate to `BUDGET_EXCEEDED` (§6.3)

5. Generate audit record of type `BUDGET_CONSUMPTION`

6. Return updated budget state

ConsumptionAcknowledgement:

```json
{
  "mandate_id": "mdt_abc123",
  "accepted": true,
  "budget": {
    "total_cents": 15000,
    "remaining_cents": 9158,
    "consumed_cents": 5842
  },
  "budget_exceeded": false
}
```

Idempotency: Consumption reports MUST include an `enforcement_request_id`. The Management API MUST reject duplicate reports with the same `enforcement_request_id` to prevent double-deduction. Duplicate reports return `accepted: false` with the current budget state.

Atomicity: Budget deduction MUST be atomic. Concurrent consumption reports for the same mandate MUST be serialized to prevent `remaining_cents` from going negative.

7.4 Extend TTL

Extends the TTL of an `ACTIVE` or `SUSPENDED` mandate. TTL extension is additive-only — the expiration time may only move forward, never backward.

Method signature:

```
extendTTL(request: TTLExtensionRequest) → TTLExtensionResponse
```

TTLExtensionRequest:

```json
{
  "mandate_id": "mdt_abc123",
  "additional_seconds": 3600,
  "extended_by": "user_789",
  "reason": "Project timeline extended"
}
```

Behavior:

1. Verify mandate is `ACTIVE` or `SUSPENDED`
2. Verify `additional_seconds` > 0
3. Verify the resulting session duration does not exceed the governance profile ceiling (`max_session_duration_minutes`)
4. Add `additional_seconds` to `expires_at` (effectively increasing the total TTL)
5. Generate audit record
6. Return updated TTL state

TTLExtensionResponse:

```json
{
  "mandate_id": "mdt_abc123",
  "previous_expires_at": "2026-04-08T22:05:00Z",
  "new_expires_at": "2026-04-08T23:05:00Z",
  "remaining_ttl_seconds": 28800,
  "audit": {
    "operation": "TTL_EXTENSION",
    "performed_by": "user_789",
    "timestamp": "2026-04-08T14:30:00Z",
    "mandate_id": "mdt_abc123",
    "details": { "additional_seconds": 3600 }
  }
}
```

Ceiling enforcement: The total effective TTL (from activation to new expiration) MUST NOT exceed the governance profile ceiling for `max_session_duration_minutes`. If the extension would exceed the ceiling, the request is rejected with `CEILING_VIOLATION`.

7.5 Query Budget

Retrieves the current budget state of a mandate.

Method signature:

```
getBudget(mandate_id: string) → BudgetState
```

BudgetState:

```json
{
  "mandate_id": "mdt_abc123",
  "status": "ACTIVE",
  "budget": {
    "total_cents": 15000,
    "remaining_cents": 9200,
    "consumed_cents": 5800,
    "utilization_percent": 38.7
  },
  "budget_history": [
    {
      "event": "INITIAL",
      "timestamp": "2026-04-08T10:00:00Z",
      "total_cents": 10000
    },
    {
      "event": "CONSUMPTION",
      "timestamp": "2026-04-08T12:30:00Z",
      "deducted_cents": 42,
      "enforcement_request_id": "enf_xyz789"
    },
    {
```

```
    "event": "INCREASE",
    "timestamp": "2026-04-08T14:30:00Z",
    "total_cents": 15000,
    "additional_cents": 5000,
    "performed_by": "user_789"
  }
 ]
}
```

---

8. MANAGEMENT API — DELEGATION LIFECYCLE

8.1 Create Delegation

Creates a delegated mandate from an existing parent mandate. The delegated mandate's scope MUST be equal to or narrower than the parent's scope (scope narrowing rule, per RFC 0117 §11).

Method signature:

```
createDelegation(request: DelegationRequest) → DelegationResponse
```

DelegationRequest:

```json
```

```
{
  "parent_mandate_id": "mdt_abc123",
  "delegate": {
    "agent_id": "agent_sub_789",
    "description": "Sub-agent for testing tasks"
  },
  "scope_restriction": {
    "allowed_paths": ["tests/"],
    "denied_paths": ["tests/fixtures/secrets/"],
    "core_verbs": {
      "file.read": { "allowed": true },
      "file.write": { "allowed": true, "constraints": { "path_patterns": ["tests/**"] } },
      "command.run": { "allowed": false }
    }
  },
  "requirements_restriction": {
    "budget": { "total_cents": 2000 },
    "ttl_seconds": 3600,
    "max_tool_calls": 100
  },
  "delegated_by": "agent_456"
}
```

Validation rules:

1. Parent mandate MUST be `ACTIVE`

2. Delegator (`delegated_by`) MUST be the parent mandate's `subject` or a member of its `approval_chain`

3. Delegation depth MUST NOT exceed `max_delegation_depth` defined in the parent's verb constraints

4. Every field in `scope_restriction` MUST be equal to or narrower than the parent's scope:

- `allowed_paths` must be a subset of or equal to parent's `allowed_paths`

- `denied_paths` may add entries but MUST NOT remove parent's `denied_paths`

- `core_verbs` may only disable verbs or add constraints — MUST NOT enable a verb the parent disables

- Platform permissions may only restrict, never expand

5. Budget MUST NOT exceed parent's `remaining_cents`

6. TTL MUST NOT exceed parent's remaining TTL

7. Governance profile MUST be equal to or stricter than parent's

Budget deduction: When a delegation is created, the delegated budget (`total_cents`) MUST be atomically deducted from the parent mandate's `remaining_cents`. This is a reservation — the parent's available budget is reduced by the amount allocated to the child. If the parent is later revoked, any unspent child budget is not returned to the parent (parent is terminal).

DelegationResponse:

```json
{
  "delegation_id": "del_xyz789",
  "child_mandate_id": "mdt_child_001",
  "parent_mandate_id": "mdt_abc123",
  "status": "ACTIVE",
  "delegation_chain": [
    {
      "delegator": "agent_456",
      "delegate": "agent_sub_789",
```

```
    "scope_restriction": { },
    "delegated_at": "2026-04-08T12:00:00Z",
    "max_depth_remaining": 0
  }
 ],
 "effective_scope": { },
 "parent_budget_after_deduction": {
  "total_cents": 10000,
  "remaining_cents": 2200,
  "consumed_cents": 5800,
  "reserved_for_delegations_cents": 2000
 },
 "audit": {
  "operation": "DELEGATE",
  "performed_by": "agent_456",
  "timestamp": "2026-04-08T12:00:00Z",
  "mandate_id": "mdt_child_001",
  "parent_mandate_id": "mdt_abc123"
 }
}
```

8.2 Revoke Delegation

Revoking a parent mandate MUST cascade to all child mandates in the delegation chain. This is a hard requirement — no orphaned active delegations may exist.

Cascade behavior:

1. When a mandate is revoked, all child mandates (direct and transitive) MUST be revoked

2. Each child receives its own audit record with `reason: "PARENT_REVOKED"`

3. Cascade is synchronous — the revocation response is not returned until all children are revoked

4. Cascade also applies to suspension: suspending a parent MUST cascade suspension to all children

8.3 Query Delegation Chain

Retrieves the full delegation chain for a mandate.

Method signature:

```
getDelegationChain(mandate_id: string) → DelegationChainResponse
```

DelegationChainResponse:

```json
{
  "mandate_id": "mdt_child_001",
  "chain": [
    {
      "level": 0,
      "mandate_id": "mdt_abc123",
      "agent_id": "agent_456",
      "role": "root",
```

```
    "status": "ACTIVE"
  },
  {
    "level": 1,
    "mandate_id": "mdt_child_001",
    "agent_id": "agent_sub_789",
    "role": "delegate",
    "status": "ACTIVE",
    "delegated_by": "agent_456",
    "delegated_at": "2026-04-08T12:00:00Z"
  }
 ],
 "effective_scope": { },
 "max_depth": 1,
 "remaining_depth": 0
}
```


---

9. MANAGEMENT API — GOVERNANCE PROFILE ASSIGNMENT

9.1 Profile Assignment

Every mandate MUST declare a governance profile. The Management API validates the mandate's parameters against the ceiling table for the declared profile.

Supported profiles (per RFC 0115):

| Profile | Description | Registration Context |

|---------|----------|----------|

| `minimal` | Least restrictive. Maximum autonomy. | Prototyping, personal projects |

| `standard` | Balanced restrictions. Supervised approval. | Small teams, general development |

| `strict` | Restrictive. Audited packages, allowlist shell. | Regulated industries |

| `enterprise` | Highly restrictive. No delegation, no secrets access. | Enterprise organizations |

| `behoerde` | Most restrictive. Four-eyes approval, 30-minute sessions. | Public sector / government |

9.2 Profile Ceiling Validation

When a mandate is created or activated, the Management API MUST validate every attribute against the governance profile ceiling:

| Attribute | Minimal | Standard | Strict | Enterprise | Behörde |

|----------|---------|----------|--------|----------|---------|

| Deployment targets | dev, staging, prod | dev, staging | staging | staging | staging |

| Auto-deploy | ✓ | ✗ | ✗ | ✗ | ✗ |

| DB write access | ✓ | ✓ | ✓ | ✗ | ✗ |

| DB migration | ✓ | ✗ | ✗ | ✗ | ✗ |

| DB production access | ✓ | ✗ | ✗ | ✗ | ✗ |

| Shell mode | any | denylist | allowlist | allowlist | allowlist |

| Packages audited only | ✗ | ✗ | ✓ | ✓ | ✓ |

| Secrets read | ✓ | ✓ | ✓ | ✗ | ✗ |

| Secrets create | ✓ | ✗ | ✗ | ✗ | ✗ |

| Agent delegation | ✓ | ✓ (depth 1) | ✓ (depth 1) | ✗ | ✗ |

| Min. approval mode | autonomous | supervised | supervised | supervised | four-eyes |

| Max. session (min) | ∞ | 240 | 120 | 60 | 30 |

| Max. tool calls | ∞ | 500 | 200 | 100 | 100 |

| Max. lines/commit | ∞ | 500 | 200 | 100 | 100 |

Validation behavior:

- Any attribute exceeding its profile ceiling produces a `CEILING_VIOLATION` error

- Multiple ceiling violations are collected and returned together (not fail-fast)

- The mandate is rejected; partial acceptance is not permitted

- Ceiling values are immutable per profile — the Management API does not allow ceiling override (proprietary extensions may offer this as an AI-assisted feature)

9.3 Profile Upgrade / Downgrade

A mandate's governance profile MUST NOT be changed after activation (scope immutability). Profile changes require a new mandate:

1. Create a new mandate with the desired profile

2. Activate the new mandate (automatically supersedes the old one)

3. The old mandate transitions to `SUPERSEDED`

In `DRAFT` state, the governance profile may be changed freely.

---

10. VALIDATION SEMANTICS

10.1 Three-Stage Validation Pipeline

Every mandate creation and activation passes through a three-stage validation pipeline. All stages are deterministic and rule-based.

```
Stage 1: Schema Validation
  │ Validate against RFC 0116 §4.3 PoA Credential Schema
  │ Result: PASS or SCHEMA_ERROR[]
  ▼
Stage 2: Ceiling Enforcement
  │ Validate each attribute against governance profile ceiling (§9.2)
  │ Result: PASS or CEILING_VIOLATION[]
  ▼
Stage 3: Consistency Checks
  │ Apply 6 deterministic consistency rules
  │ Result: PASS or CONSISTENCY_ERROR[]
  ▼
Overall: ACCEPTED (all three stages pass) or REJECTED (structured errors from all stages)
```

10.2 Stage 1 — Schema Validation

The mandate's `scope` and `requirements` objects are validated against the canonical PoA JSON Schema (RFC 0116 §4.3, `https://gimelfoundation.com/schemas/poa/v2.2/poa-credential.json`).

Checks:

- All required fields present
- Field types match schema definitions
- Enum values are valid members

- `core_verbs` entries conform to the `ToolPolicy` schema

- `platform_permissions` conform to the `PlatformPermissions` schema

- `governance_profile` is a recognized profile name (reject with `PROFILE_NOT_FOUND` if unknown)

10.3 Stage 2 — Ceiling Enforcement

Each of the 14 ceiling attributes (7 stateless + 7 stateful) is compared against the governance profile ceiling table (§9.2).

Checks:

- Session duration ≤ profile maximum

- Tool call limit ≤ profile maximum

- Lines per commit ≤ profile maximum

- Approval mode meets profile minimum (e.g., `behoerde` requires `four-eyes`)

- Deployment targets are subset of profile allowed targets

- Platform permissions do not exceed profile allowances

- Delegation depth ≤ profile maximum

10.4 Stage 3 — Consistency Checks

Six deterministic consistency rules are applied:

Rule	Check	Error Code
C-1	`four-eyes` approval mode requires `approval_chain` with ≥ 2 distinct entries	`FOUR_EYES_MISSING_APPROVERS`
C-2	`denied_paths` and `allowed_paths` must not contain the same entry	`PATH_CONFLICT`

| C-3 | Budget `total_cents` must be ≥ 0 (zero-budget mandates are valid for read-only agents) | `INVALID_BUDGET` |

| C-4 | TTL must be ≥ 60 seconds | `TTL_TOO_SHORT` |

| C-5 | If `active_modules` references a module, that module must exist in the governance profile's module registry | `UNKNOWN_MODULE` |

| C-6 | If `delegation_chain` is present (for delegated mandates), every delegation entry must satisfy scope narrowing | `DELEGATION_SCOPE_WIDENING` |

10.5 Validation Response

When validation fails, the Management API returns a structured error listing all violations across all three stages:

```json
{
  "accepted": false,
  "validation": {
    "schema_errors": [
      {
        "path": "$.scope.core_verbs.file.modify.allowed",
        "error": "Expected boolean, got string",
        "code": "SCHEMA_TYPE_MISMATCH"
      }
    ],
    "ceiling_violations": [
      {
        "attribute": "max_session_duration_minutes",
        "requested": 480,
        "ceiling": 240,
        "profile": "standard",
```

```
      "code": "CEILING_VIOLATION"
    }
  ],
  "consistency_errors": [
    {
      "rule": "C-1",
      "message": "four-eyes approval mode requires approval_chain with >= 2 distinct entries",
      "code": "FOUR_EYES_MISSING_APPROVERS"
    }
  ]
 }
}
```

---

11. HTTP BINDING

11.1 Endpoint Table

The Management API is exposed as a RESTful HTTP API:

Method	Path	Request Body	Response	Description
`POST`	`/gauth/mgmt/v1/mandates`	`MandateCreationRequest`	`MandateCreationResponse`	Create mandate (DRAFT)
`POST`	`/gauth/mgmt/v1/mandates/{id}/activate`	`MandateActivationRequest`	`MandateActivationResponse`	Activate mandate (DRAFT → ACTIVE)

| `POST` | `/gauth/mgmt/v1/mandates/{id}/revoke` | `MandateRevocationRequest` | `MandateRevocationResponse` | Revoke mandate |

| `POST` | `/gauth/mgmt/v1/mandates/{id}/suspend` | `MandateSuspensionRequest` | `MandateSuspensionResponse` | Suspend mandate (ACTIVE → SUSPENDED) |

| `POST` | `/gauth/mgmt/v1/mandates/{id}/resume` | `MandateResumptionRequest` | `MandateResumptionResponse` | Resume mandate (SUSPENDED → ACTIVE) |

| `DELETE` | `/gauth/mgmt/v1/mandates/{id}` | — | `DeletionConfirmation` | Delete DRAFT mandate |

| `GET` | `/gauth/mgmt/v1/mandates/{id}` | — | `MandateDetail` | Get mandate |

| `GET` | `/gauth/mgmt/v1/mandates` | Query params | `MandateListResponse` | List mandates |

| `GET` | `/gauth/mgmt/v1/mandates/{id}/history` | — | `MandateHistoryResponse` | Get mandate audit trail |

| `POST` | `/gauth/mgmt/v1/mandates/{id}/budget/increase` | `BudgetIncreaseRequest` | `BudgetIncreaseResponse` | Increase budget ceiling |

| `POST` | `/gauth/mgmt/v1/mandates/{id}/budget/consume` | `ConsumptionReport` | `ConsumptionAcknowledgement` | Report budget consumption (PEP → Management API) |

| `POST` | `/gauth/mgmt/v1/mandates/{id}/ttl/extend` | `TTLExtensionRequest` | `TTLExtensionResponse` | Extend mandate TTL |

| `GET` | `/gauth/mgmt/v1/mandates/{id}/budget` | — | `BudgetState` | Get budget state |

| `POST` | `/gauth/mgmt/v1/delegations` | `DelegationRequest` | `DelegationResponse` | Create delegation |

| `GET` | `/gauth/mgmt/v1/mandates/{id}/delegation-chain` | — | `DelegationChainResponse` | Get delegation chain |

| `GET` | `/gauth/mgmt/v1/profiles` | — | `ProfileListResponse` | List available governance profiles |

| `GET` | `/gauth/mgmt/v1/profiles/{profile}/ceilings` | — | `CeilingTable` | Get ceiling table for a profile |

| `GET` | `/gauth/mgmt/v1/health` | — | `{ status, mgmt_version, interface_version }` | Health and version discovery |

11.2 Authentication

All Management API endpoints MUST require authentication. The specification does not mandate a specific authentication mechanism — implementations may use:

- OAuth 2.1 bearer tokens
- Mutual TLS (mTLS)
- API keys with HMAC signatures
- Platform-specific S2S authentication (e.g., Platform Key + HMAC as used by GAuth internally)

The caller's identity MUST be captured in audit records.

Budget consumption endpoint authentication: The `/budget/consume` endpoint is called by PEP enforcement systems. Implementations SHOULD use a distinct authentication scope or credential for PEP-to-Management-API calls to distinguish enforcement-initiated budget deductions from administrative operations in audit trails.

11.3 Request/Response Format

- Content-Type: `application/json`
- Character encoding: UTF-8
- Timestamps: ISO 8601 with UTC timezone (`2026-04-08T10:00:00Z`)
- Pagination: Cursor-based (opaque cursor string)
- Error responses: Use the error schema defined in Section 12

11.4 Rate Limiting

Management API implementations SHOULD implement rate limiting to prevent abuse. Recommended limits:

| Operation Category | Recommended Limit |

|---|---|

| Create / Activate / Revoke / Suspend / Resume | 100 requests per minute per caller |

| Query / List / Get | 1000 requests per minute per caller |

| Budget increase / TTL extension | 50 requests per minute per caller |

| Budget consumption reports | 10000 requests per minute per PEP (high-throughput enforcement) |

| Delegation creation | 50 requests per minute per caller |

Rate limit responses use HTTP 429 with the error schema (Section 12).

---

12. ERROR RESPONSE SCHEMA

12.1 ManagementError Schema

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://gimelfoundation.com/schemas/mgmt/v1.1/management-error.json",
  "title": "GAuth Management API Error Response",
  "type": "object",
  "required": ["error_code", "message", "timestamp"],
  "properties": {
    "error_code": {
      "type": "string",
      "enum": [
        "MANDATE_NOT_FOUND",
```

```
      "PROFILE_NOT_FOUND",
      "INVALID_STATE_TRANSITION",
      "SCHEMA_VALIDATION_FAILED",
      "CEILING_VIOLATION",
      "CONSISTENCY_CHECK_FAILED",
      "INSUFFICIENT_AUTHORITY",
      "MANDATE_NOT_DRAFT",
      "MANDATE_NOT_ACTIVE",
      "MANDATE_NOT_SUSPENDED",
      "MANDATE_EXPIRED",
      "BUDGET_DECREASE_NOT_ALLOWED",
      "TTL_DECREASE_NOT_ALLOWED",
      "DELEGATION_DEPTH_EXCEEDED",
      "DELEGATION_SCOPE_WIDENING",
      "DELEGATION_BUDGET_EXCEEDED",
      "DELEGATION_TTL_EXCEEDED",
      "PARENT_MANDATE_NOT_ACTIVE",
      "DUPLICATE_MANDATE",
      "DUPLICATE_CONSUMPTION_REPORT",
      "RATE_LIMITED",
      "INTERNAL_ERROR"
    ]
  },
  "message": {
    "type": "string",
    "description": "Human-readable error description"
  },
  "timestamp": {
    "type": "string",
```

```
    "format": "date-time"
  },
  "details": {
    "type": "object",
    "description": "Structured error details (validation errors, ceiling violations, etc.)"
  },
  "request_id": {
    "type": "string",
    "description": "Correlation ID for tracing"
  },
  "retry_after_seconds": {
    "type": "integer",
    "description": "For RATE_LIMITED errors: recommended wait time before retry"
  }
 }
}
```

12.2 HTTP Status Code Mapping

Error Code	HTTP Status	Description
`MANDATE_NOT_FOUND`	404	Mandate ID does not exist
`PROFILE_NOT_FOUND`	404	Governance profile name is not recognized
`INVALID_STATE_TRANSITION`	409	Requested transition is not valid from current state
`SCHEMA_VALIDATION_FAILED`	422	Request body fails PoA schema validation
`CEILING_VIOLATION`	422	One or more attributes exceed governance profile ceiling

| `CONSISTENCY_CHECK_FAILED` | 422 | One or more consistency rules violated |

| `INSUFFICIENT_AUTHORITY` | 403 | Caller does not have authority for this operation |

| `MANDATE_NOT_DRAFT` | 409 | Operation requires DRAFT state |

| `MANDATE_NOT_ACTIVE` | 409 | Operation requires ACTIVE state |

| `MANDATE_NOT_SUSPENDED` | 409 | Operation requires SUSPENDED state (e.g., resume) |

| `MANDATE_EXPIRED` | 409 | Mandate has expired — TTL elapsed. Distinct from MANDATE_NOT_ACTIVE to enable precise error handling |

| `BUDGET_DECREASE_NOT_ALLOWED` | 422 | Budget ceiling may only increase (additive-only rule) |

| `TTL_DECREASE_NOT_ALLOWED` | 422 | TTL may only be extended (additive-only rule) |

| `DELEGATION_DEPTH_EXCEEDED` | 422 | Delegation chain would exceed maximum depth |

| `DELEGATION_SCOPE_WIDENING` | 422 | Delegated scope is wider than parent scope |

| `DELEGATION_BUDGET_EXCEEDED` | 422 | Delegated budget exceeds parent's remaining budget |

| `DELEGATION_TTL_EXCEEDED` | 422 | Delegated TTL exceeds parent's remaining TTL |

| `PARENT_MANDATE_NOT_ACTIVE` | 409 | Parent mandate is not in ACTIVE state |

| `DUPLICATE_MANDATE` | 409 | An active mandate already exists for this (agent, project) pair |

| `DUPLICATE_CONSUMPTION_REPORT` | 409 | A consumption report with this enforcement_request_id has already been processed |

| `RATE_LIMITED` | 429 | Rate limit exceeded |

| `INTERNAL_ERROR` | 500 | Unexpected internal error |

---

13. INTERFACE VERSIONING

13.1 Version Identifier

The Management API uses a major version in the URL path (`/gauth/mgmt/v1/`). The full interface version is reported by the health endpoint:

```json
{
  "status": "ok",
  "mgmt_version": "1.1.0",
  "interface_version": "1.1",
  "supported_schema_version": "0116.2.2",
  "features": {
    "suspension": true,
    "ttl_extension": true,
    "budget_consumption_reporting": true
  }
}
```

13.2 Compatibility Rules

- **Minor version increments** (1.0 → 1.1): Backward-compatible additions (new optional fields, new endpoints). Existing clients continue to work.
- **Major version increments** (v1 → v2): Breaking changes. New URL path prefix (`/gauth/mgmt/v2/`). Old version remains available during deprecation period.
- **Schema version dependency:** The Management API validates against a specific version of the RFC 0116 PoA schema. The `supported_schema_version` field in the health response indicates which schema version is expected.

---

14. ADMINISTRATION INTERFACE PATTERNS (NON-NORMATIVE)

This section provides non-normative guidance for building administration UIs on top of the Management API. These are functional requirements, not visual prescriptions — implementations should adapt layouts and styling to their target audience.

14.1 Mandate Creation Flow

A mandate creation UI SHOULD implement a multi-step flow with ceiling-aware validation:

1. **Agent & Project Selection** — Select or search for the target agent and project. Auto-fill issuer from session.

2. **Governance Profile Selection** — Select profile; display ceiling table preview on selection; disable options that would violate the selected profile's ceilings.

3. **Scope Configuration** — Phase selection, allowed/denied paths, module selection, core verb configuration with per-verb toggles and constraints, platform permission toggles.

4. **Requirements** — Approval mode, budget, TTL, session limits. All fields constrained by the selected profile's ceilings.

5. **Review & Create** — Full mandate preview (structured and/or JSON), validation status indicator, choice to create as DRAFT or create and activate immediately.

Key UX principles:

- Profile selection SHOULD constrain available options in subsequent steps (ceiling-aware form)

- Real-time validation feedback as the user fills in fields

- Ceiling violations shown inline (not only on submission)

14.2 Mandate Inspection View

A mandate inspection UI SHOULD include:

| Component | Content |

|----------|---------|

| **Header** | Mandate ID, status badge (including SUSPENDED), agent name, project name |

| **Scope panel** | Governance profile, phase, paths, modules, verbs (expandable), platform permissions |

| **Requirements panel** | Approval mode, budget utilization, TTL remaining, session limits |

| **Delegation panel** | Delegation chain visualization, effective scope after narrowing |

| **History panel** | Chronological audit trail (create, activate, suspend, resume, budget changes, delegation, revocation) |

| **Actions** | Suspend/Resume toggle, Revoke (with confirmation), Budget increase, TTL extension, Delegate |

14.3 Mandate List View

A mandate list UI SHOULD display: Mandate ID, Agent, Project, Governance Profile, Status (color-coded, including SUSPENDED), Budget utilization, TTL remaining, and Creation date. Support filtering by status (including SUSPENDED), profile, agent, project, and date range.

14.4 Budget Monitoring

A budget monitoring UI SHOULD provide: per-mandate budget utilization display, consumption history, alerts for mandates approaching budget exhaustion (>80% consumed), and a view of top-consuming agents.

14.5 Delegation Management

A delegation management UI SHOULD provide: delegation chain visualization (tree or hierarchy), side-by-side scope comparison (parent vs. delegated scope highlighting restrictions), cascade warning for revocation/suspension, and a delegation creation form pre-populated with parent scope as maximum bounds.

14.6 Governance Profile Browser

A governance profile UI SHOULD provide: all 5 profiles displayed with ceiling values for comparison, and an impact preview showing which current DRAFT settings would violate a new profile's ceilings.

---

15. CONFORMANCE REQUIREMENTS

15.1 Conformant Management API

A conformant Management API implementation MUST:

1. Implement all endpoints defined in Section 11.1
2. Validate all mandates through the three-stage pipeline (Section 10.1)
3. Enforce all governance profile ceilings (Section 9.2) without exception or override
4. Enforce all status transition rules (Section 6.1) — no invalid transitions
5. Enforce scope immutability for ACTIVE and SUSPENDED mandates (Section 3.3)
6. Enforce additive-only budget ceiling and TTL changes (Sections 7.2, 7.4)
7. Enforce delegation scope narrowing (Section 8.1)
8. Enforce delegation budget deduction atomically from parent mandate (Section 8.1)
9. Enforce supersession atomicity (Section 6.4)
10. Accept and process budget consumption reports with idempotency (Section 7.3)
11. Generate audit records for all management operations

12. Return structured errors conforming to Section 12

13. Report interface version and feature support via the health endpoint (Section 13.1)

15.2 Conformant Administration Interface

A conformant Administration Interface implementation SHOULD:

1. Use the Management API (Section 11) as its sole data layer — no direct mandate store access

2. Display ceiling constraints during mandate creation (ceiling-aware forms)

3. Show real-time validation feedback

4. Display delegation chains with scope comparison

5. Provide budget monitoring with utilization visualization

6. Enforce confirmation dialogs for destructive operations (revocation, deletion)

7. Display SUSPENDED state distinctly from ACTIVE and terminal states

Note: Administration Interface conformance is SHOULD (non-normative), not MUST.

---

16. SECURITY CONSIDERATIONS

16.1 Authentication and Authorization

- All Management API endpoints MUST require authentication (Section 11.2)

- Mandate creation authority SHOULD be restricted to designated administrators or provisioning systems

- Revocation and suspension authority MUST be limited to the original issuer (`issued_by`) or members of the `approval_chain`

- Delegation authority MUST be limited to the current mandate holder (the agent) or the mandate issuer

- Budget increase and TTL extension authority SHOULD be restricted to the mandate issuer or organizational administrators

- Budget consumption reporting authority SHOULD be restricted to authenticated PEP systems

16.2 Audit Trail Integrity

- Audit records MUST be append-only — no modification or deletion of audit entries

- Audit records MUST include the identity of the caller, the operation performed, the timestamp, and the affected mandate

- Budget consumption audit records MUST include the `enforcement_request_id` for traceability to PEP decisions

- For regulatory environments (particularly `behoerde` profile), audit retention SHOULD comply with applicable data retention requirements

16.3 Fail-Closed Validation

- If the Management API cannot complete validation (schema resolution failure, ceiling data unavailable, internal error), the mandate MUST be rejected

- There is no auto-approve mode, no bypass mechanism, and no administrative override for validation failures

- This is consistent with the fail-closed principle in RFC 0117 §1.2

16.4 Scope Immutability Enforcement

- Once a mandate transitions from DRAFT to ACTIVE, scope MUST be immutable at the application layer

- Scope MUST remain immutable during SUSPENDED state

- Implementations SHOULD also enforce immutability at the data store layer (e.g., database triggers, immutable columns)

- Any attempt to modify scope on an ACTIVE or SUSPENDED mandate MUST return `MANDATE_NOT_DRAFT` error

16.5 Delegation Chain Security

- Scope narrowing MUST be enforced — a delegation MUST NOT expand the parent's scope

- Budget allocation from parent to child MUST be deducted from the parent's `remaining_cents` atomically at delegation creation time — no double-spending

- Delegation depth limits MUST be enforced at creation time, not only at enforcement time

- Cascade revocation MUST be synchronous — no window where a child mandate is active after its parent is revoked

- Cascade suspension MUST be synchronous — no window where a child mandate is active after its parent is suspended

16.6 Rate Limiting

- Management API implementations SHOULD implement rate limiting (Section 11.4)

- Rate limiting prevents abuse of the validation pipeline (schema validation + ceiling enforcement can be computationally expensive)

- The budget consumption endpoint SHOULD have a higher rate limit than administrative endpoints to accommodate high-throughput PEP enforcement

- Rate limit responses MUST use the standard error schema with `RATE_LIMITED` code

16.7 Input Validation

- All string inputs MUST be validated for maximum length (implementation-defined limits)

- Path patterns (`allowed_paths`, `denied_paths`) MUST be validated for glob syntax safety — no path traversal attacks

- Mandate IDs MUST be unpredictable (UUID v4, ULID, or equivalent) — no sequential IDs

- `enforcement_request_id` values in consumption reports MUST be validated for uniqueness (idempotency key)

16.8 Supersession Safety

- The atomicity requirements in §6.4 MUST be enforced to prevent the "dual active mandate" race condition

- Implementations MUST use database-level locking or compare-and-swap to guarantee that at most one mandate is ACTIVE for any given `(agent_id, project_id)` pair at any time

---

Appendix A: Reference Implementation Pseudocode

A.1 Create Mandate

```
function createMandate(request):
  // Stage 1: Schema validation
  schema_errors = validateAgainstSchema(request, POA_CREDENTIAL_SCHEMA)
  if schema_errors:
    return REJECTED(schema_errors)

  // Check governance profile exists
  profile = request.scope.governance_profile
  if not profileExists(profile):
    return ERROR(PROFILE_NOT_FOUND, profile)
```

```
// Stage 2: Ceiling enforcement
ceilings = getCeilingTable(profile)
ceiling_violations = []

if request.requirements.session_limits.max_session_duration_minutes >
ceilings.max_session_minutes:
    ceiling_violations.append(CEILING_VIOLATION("max_session_duration_minutes",
...))
if request.requirements.session_limits.max_tool_calls > ceilings.max_tool_calls:
    ceiling_violations.append(CEILING_VIOLATION("max_tool_calls", ...))
if request.requirements.session_limits.max_lines_per_commit >
ceilings.max_lines_per_commit:
    ceiling_violations.append(CEILING_VIOLATION("max_lines_per_commit", ...))
if not meetsMinimumApprovalMode(request.requirements.approval_mode,
ceilings.min_approval_mode):
    ceiling_violations.append(CEILING_VIOLATION("approval_mode", ...))
// ... validate all 14 attributes

if ceiling_violations:
    return REJECTED(ceiling_violations)

// Stage 3: Consistency checks
consistency_errors = []
if request.requirements.approval_mode == "four-eyes":
    if not request.parties.approval_chain or len(request.parties.approval_chain) < 2:
        consistency_errors.append(CONSISTENCY_ERROR("C-1",
"FOUR_EYES_MISSING_APPROVERS"))

overlap = intersection(request.scope.allowed_paths, request.scope.denied_paths)
```

```
if overlap:
  consistency_errors.append(CONSISTENCY_ERROR("C-2", "PATH_CONFLICT"))

if request.requirements.budget.total_cents < 0:
  consistency_errors.append(CONSISTENCY_ERROR("C-3", "INVALID_BUDGET"))

if request.requirements.ttl_seconds < 60:
  consistency_errors.append(CONSISTENCY_ERROR("C-4", "TTL_TOO_SHORT"))

// ... remaining consistency checks

if consistency_errors:
  return REJECTED(consistency_errors)

// All stages passed — create mandate
mandate = Mandate(
  id = generateMandateId(),  // UUID v4, unpredictable
  status = DRAFT,
  parties = request.parties,
  scope = request.scope,
  requirements = request.requirements,
  scope_checksum = computeScopeChecksum(request.scope),
  tool_permissions_hash = computeHash(request.scope.core_verbs),
  platform_permissions_hash = computeHash(request.scope.platform_permissions),
  created_at = now()
)

store(mandate)
auditLog(CREATE, mandate.id, request.parties.issued_by)
```

```
  return ACCEPTED(mandate)
```

A.2 Create Delegation

```
function createDelegation(request):
  parent = getMandate(request.parent_mandate_id)
  if parent.status != ACTIVE:
    return ERROR(PARENT_MANDATE_NOT_ACTIVE)

  // Verify delegation authority
  if request.delegated_by != parent.parties.subject
    and request.delegated_by not in parent.parties.approval_chain:
    return ERROR(INSUFFICIENT_AUTHORITY)

  // Check delegation depth
  current_depth = len(parent.delegation_chain)
  max_depth = getMaxDelegationDepth(parent.scope)
  if current_depth >= max_depth:
    return ERROR(DELEGATION_DEPTH_EXCEEDED)

  // Verify scope narrowing (two-pass, per RFC 0117 §11)
  if not isScopeSubset(request.scope_restriction, parent.scope):
    return ERROR(DELEGATION_SCOPE_WIDENING)

  // Verify budget
  if request.requirements_restriction.budget.total_cents >
parent.requirements.budget.remaining_cents:
```

```
    return ERROR(DELEGATION_BUDGET_EXCEEDED)

  // Verify TTL
  parent_remaining_ttl = parent.expires_at - now()
  if request.requirements_restriction.ttl_seconds > parent_remaining_ttl:
    return ERROR(DELEGATION_TTL_EXCEEDED)

  // Atomically: deduct budget from parent + create child mandate
  atomicTransaction:
    deductBudget(parent, request.requirements_restriction.budget.total_cents)
    child_mandate = createMandate(buildDelegatedMandateRequest(parent, request))
    child_mandate.delegation_chain = parent.delegation_chain +
[newDelegationEntry(request)]
    activateMandate(child_mandate.id)  // delegations skip DRAFT

  auditLog(DELEGATE, child_mandate.id, request.delegated_by, parent.id)
  return DelegationResponse(child_mandate)
```

A.3 Report Consumption

```
function reportConsumption(request):
  // Idempotency check
  if consumptionAlreadyProcessed(request.enforcement_request_id):
    return ConsumptionAcknowledgement(accepted=false,
budget=getCurrentBudget(request.mandate_id))

  mandate = getMandate(request.mandate_id)
```

```
  if mandate.status != ACTIVE:
    return ERROR(MANDATE_NOT_ACTIVE)

  // Atomic budget deduction
  atomicTransaction:
    mandate.requirements.budget.remaining_cents -= request.deducted_cents
    if mandate.requirements.budget.remaining_cents <= 0:
      mandate.requirements.budget.remaining_cents = 0
      mandate.status = BUDGET_EXCEEDED
      auditLog(STATUS_TRANSITION, mandate.id, "system", "BUDGET_EXCEEDED")
    recordConsumption(request.enforcement_request_id, request.deducted_cents)

  auditLog(BUDGET_CONSUMPTION, mandate.id, request.reported_by,
request.enforcement_request_id)
  return ConsumptionAcknowledgement(accepted=true,
budget=mandate.requirements.budget)
```

Appendix B: Management API Schema Registry

The Management API introduces the following schema URIs:

URI	Description
`https://gimelfoundation.com/schemas/mgmt/v1.1/mandate-creation-request.json`	MandateCreationRequest schema (Section 4.1)
`https://gimelfoundation.com/schemas/mgmt/v1.1/management-error.json`	ManagementError schema (Section 12.1)

These schemas are published alongside the RFC 0116 PoA schemas and the RFC 0117 PEP schemas. Implementations SHOULD validate requests and responses against these schemas.

Appendix C: Administration Interface Wireframe Descriptions

The wireframe descriptions in Section 14 are intentionally text-based rather than graphical. This allows:

1. Implementation in any UI framework without prescribing a visual style
2. Accessibility review at the specification level
3. Translation to different form factors (desktop, tablet, mobile)

Implementations are encouraged to conduct usability testing with their target audience and adapt the layouts accordingly while maintaining the functional requirements (ceiling-aware forms, real-time validation, delegation visualization).

Appendix D: Open Questions

#	Question	Context	Auth Team Position (v1.1)
OQ-1	Should the Management API support bulk mandate creation?	Provisioning systems may need to create mandates for many agents simultaneously	Recommend for v1.2. CodeShield onboards teams with multiple agents.
OQ-2	Should mandate templates be a first-class concept in the Management API?	Templates would allow reusable scope configurations, reducing creation friction

Recommend for v1.2. Governance profiles already function as implicit templates; making them explicit reduces friction. |

| OQ-3 | How should the Management API interact with the AI-assisted registration paths (1–3)? | Should Path 4 be a fallback when AI paths are unavailable, or a parallel option? | Path 4 is the **foundation**, not a fallback. Paths 1–3 layer AI enhancement on top. Path 4 validation always runs; Paths 1–3 add risk scoring and regulatory analysis. |

| OQ-4 | Should the Management API support mandate cloning (create a new DRAFT from an existing mandate's scope)? | Common workflow for renewal scenarios | Recommend for v1.2. Useful for TTL-based renewal workflows where the scope should be preserved. |

| OQ-6 | Should the Management API support webhook notifications for status transitions? | Enables reactive systems (e.g., alert on revocation, notify on budget exhaustion) | Recommend for v1.2. Auth already has webhook infrastructure. |

| OQ-7 | Should the Administration Interface specification include role-based access control (RBAC) patterns? | Different UI views for administrators, project leads, and auditors | The Management API itself should define who can do what (create/revoke/delegate authority) normatively. RBAC for the Admin UI is implementation-specific. |

| OQ-8 | How should the Management API handle cross-platform mandate portability? | A mandate created on Platform A may need to be recognized by Platform B | Deferred. This is RFC 0116 territory — the token format enables portability. The Management API is local to each platform. |

Resolved in v1.1:

| # | Resolution |

|---|----------|

| OQ-5 (Budget consumption reporting) | **Resolved.** Section 7.3 defines the `reportConsumption()` interface and the `/budget/consume` endpoint. PEP implementations report consumption to the Management API with idempotent enforcement request IDs. |

Appendix E: Reference Documents

| Document | Version | Relevance |

|----------|---------|-----------|

| GiFo-RFC 0110 | — | GAuth Protocol Engine (transport, routing) |

| GiFo-RFC 0111 | — | Authorization Framework (flows, roles, trust model) |

| GiFo-RFC 0115 | — | Three-Layer Capability Model; governance profile definitions and ceiling tables |

| GiFo-RFC 0116 | 2.2 | Interoperability Layer; PoA Credential Schema (§4), Token Lifecycle (§10), Error Response Schema (§12) |

| GiFo-RFC 0117 | 1.2 | PEP Interface Contract; evaluation semantics (§9), enforcement modes (§10), delegation chain (§11) |

| AGENT_MANDATE_SPEC_V03 | 0.3 | Mandate schema, governance profile ceiling table, validation logic, risk scoring |

Appendix F: Revision History

| Version | Date | Changes |

|---------|------|---------|

| 1.0 | 2026-04-08 | Initial draft by Architecture Working Group. Management API contract (mandate CRUD, delegation lifecycle, budget operations, governance profile assignment), three-stage validation pipeline, HTTP binding, error response schema, Administration Interface patterns (non-normative). |

| 1.1 | 2026-04-09 | Auth Team review. **(1) SUSPENDED state** — Added SUSPENDED lifecycle state with suspend/resume operations (§3.2, §4.4, §4.5, §6.1). Cascade suspension to delegation chain. TTL clock continues during suspension. **(2) Supersession atomicity** — Added §6.4 requiring atomic compare-and-swap for supersession transitions; no dual-active-mandate window (§16.8). **(3) Budget consumption reporting** — Resolved OQ-5: added `reportConsumption()` interface (§7.3) and `/budget/consume` endpoint (§11.1) with idempotency via `enforcement_request_id`. PEP → Management API budget sync is now normative. **(4) TTL extension** — Added `extendTTL()` operation (§7.4) and `/ttl/extend` endpoint with additive-only semantics and ceiling enforcement. **(5) Delegation budget deduction**

— Made budget deduction from parent's `remaining_cents` normative in §8.1 (previously only in pseudocode and security considerations). **(6) New error codes** — Added `PROFILE_NOT_FOUND`, `MANDATE_EXPIRED`, `MANDATE_NOT_SUSPENDED`, `TTL_DECREASE_NOT_ALLOWED`, `DUPLICATE_CONSUMPTION_REPORT` (§12). **(7) Domain consistency** — Fixed example issuer from `gauth.gimel.dev` to `gimelfoundation.com` (§5.1). **(8) Path 4 positioning** — Clarified that Path 4 is the foundation, not a fallback; Paths 1–3 layer AI on top (§1.6, Appendix D OQ-3). **(9) Administration Interface** — Condensed §14 to functional requirements; removed prescriptive UI component recommendations. **(10) Schema URIs** — Updated to v1.1 namespace (`/mgmt/v1.1/`). |

Disclaimer: ALL DOCUMENTS AND THE INFORMATION CONTAINED THEREIN ARE PROVIDED ON AN “AS IS” BASIS AND THE CONTRIBUTOR, THE ORGANIZATION THEY REPRESENT OR ARE SPONSORED BY (IF ANY), THE GIMEL FOUNDATION, AND ANY APPLICABLE MANAGERS OF ALTERNATE DOCUMENT STREAMS, DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION THEREIN WILL NOT INFRINGE ANY RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

* * *